

DEDICATED TO INNOVATION

www.bacancy.com





**Customer Satisfaction
Is Our Highest Priority**



Employee Matters



In this session

What we are learning in this session

- ▶ EF Development Approach
- ▶ Introduction to EF Entities Relationships
- ▶ One to One Relationship
- ▶ One to Many Relationship
- ▶ Many to Many Relationship

Let's Start

EF - Development Approach

- ▶ There are three different approaches you can use while developing your application using Entity Framework:
 - Database-First
 - Code-First
 - Model-First

EF - Database-First Approach

- ▶ In the database-first development approach, you generate the context and entities for the existing database using EDM wizard integrated in Visual Studio or executing EF commands.

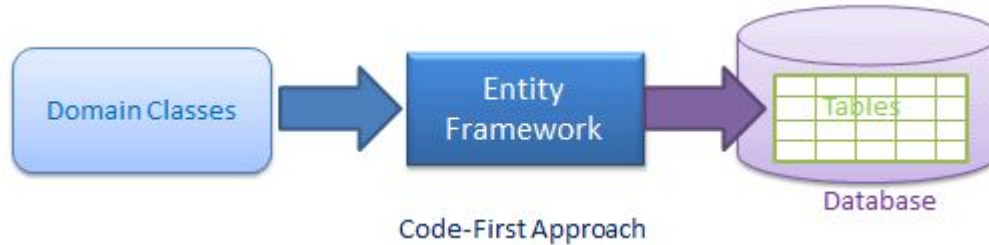


Database-First Approach

- ▶ EF 6 supports the database-first approach extensively. Visit EF 6 DB-First section to learn about the database-first approach using EF 6.
- ▶ EF Core includes limited support for this approach.

EF - Code-First Approach

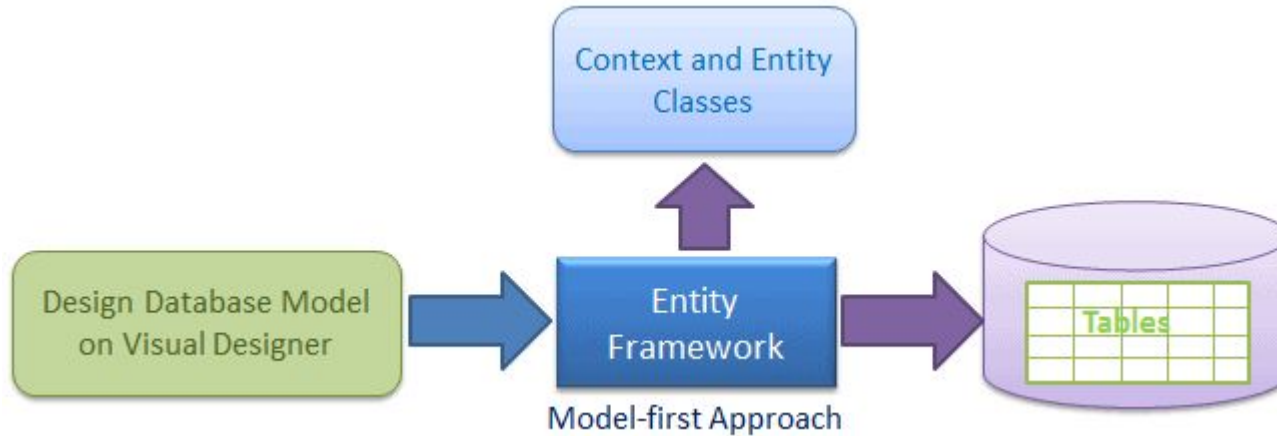
- ▶ Use this approach when you do not have an existing database for your application. In the code-first approach, you start writing your entities (domain classes) and context class first and then create the database from these classes using migration commands.



- ▶ Developers who follow the Domain-Driven Design (DDD) principles, prefer to begin with coding their domain classes first and then generate the database required to persist their data.

EF - Model-First Approach

- ▶ In the model-first approach, you create entities, relationships, and inheritance hierarchies directly on the visual designer integrated in Visual Studio and then generate entities, the context class, and the database script from your visual model.



- ▶ EF 6 includes limited support for this approach.
- ▶ EF Core does not support this approach.

Relationships between Entities

- ▶ In relational databases, relationships between tables (or entities) define how data in one table relates to data in another table.
- ▶ Relationships or associations between entities are defined in a database by the existence of Foreign Keys.
- ▶ When we create relationships between two entities in Entity Framework Core, one entity serves as the **Principal Entity**, and the other entity serves as the **Dependent Entity**.

Relationships between Entities

- ▶ **Navigation Properties:** Navigation Properties in entity classes allow users to navigate from one entity to a related entity or collection of entities.

Reference Navigation Property: It represents a single related entity, typically in a one-to-one relationship. For example, a Customer might have exactly one Address.

Collection Navigation Property: It represents a collection of related entities commonly used in one-to-many or many-to-many relationships. For example, a Customer might have many Orders.

Types of Relationships

- ▶ **One-to-One (1:1):** A one-to-one (or more usually a one-to-zero or one) relationship exists when only one row of data in the principal table is linked to zero or one row in a dependent table.
- ▶ **One-to-Many:** In a one-to-many relationship, one entity instance (the principal) relates to multiple instances of another entity (the dependents). Typically, the principal entity holds the primary key, while the dependent entity holds the foreign key.
- ▶ **Many-to-Many:** In this relationship, both entities have collection navigation properties, and EF Core creates an implicit join table in the database to link the two.

One to One Relationships

```
public class Student
{
    [Key]
    public int StudentId { get; set; }
    public string Name { get; set; }

    public StudentAddress Address { get; set; }
}
```

```
public class StudentAddress
{
    public int Id { get; set; }
    public string Street { get; set; }
    public string City { get; set; }

    // Foreign Key
    [ForeignKey("StudentId")]
    public int StudentId { get; set; }

    // Navigation Property
    public Student Student { get; set; }
}
```

One to Many Relationships

```
public class Course
{
    public int Id { get; set; }
    public string Name { get; set; }

    // Navigation property (One Course can have many Students)
    public List<Student> Students { get; set; } = new List<Student>();
}
```

```
public class Student
{
    [Key]
    public int StudentId { get; set; }
    public string Name { get; set; }

    public StudentAddress Address { get; set; }

    [ForeignKey("CourseId")]
    public int? CourseId { get; set; }

    public Course course { get; set; }
}
```

Many to Many Relationships

```
public class Student
{
    public int Id { get; set; }
    public string Name { get; set; }

    // Many-to-Many relationship with Clubs
    public List<Club> Clubs { get; set; } = new List<Club>();
}
```

```
public class Club
{
    public int Id { get; set; }
    public string Name { get; set; }

    // Many-to-Many relationship with Students
    public List<Student> Students { get; set; } = new List<Student>();
}
```

Thank you